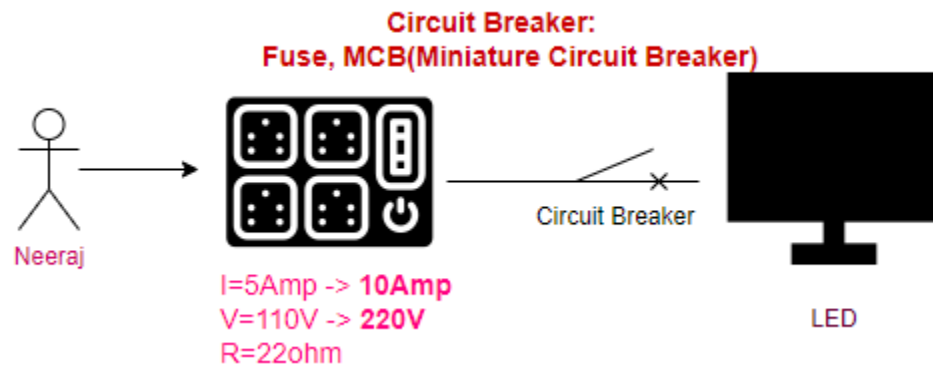
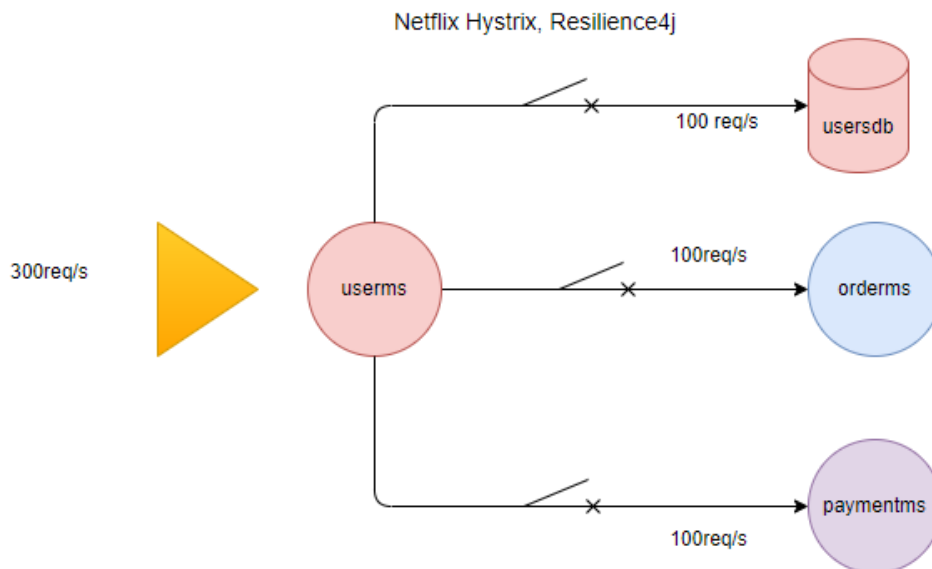


UC: Voltage Increase from 110V->220V, due to which LED may be crashed.
USB will break the circuit, will save LED from crash.



Circuit Breaker (applies where there is a network call) for Resiliency:
users getting 300 req/sec (100req/sec->ordersms, 100req/sec->paymentms, 100req/sec->db), ordersms is crashed, usersms retries and after 5 minutes it retries 3000 req/sec. most of the **resources busy** on retries, will impact the performance of whole application.

Circuit Breaker (applies where there is a network call) for Resiliency:



Solution: open the circuit of down service (ordersms) after 3 consecutive tries, now usersms will not retries.

In background continue checks the heart beating once get again (after 3 consecutive calls) close the circuit again.

Resiliency:

Failure in one component can affect to other components and after sometime intern fail the whole application. Failure don't cascade into other parts of application. This is not a resilience application.

Solution: Circuit Breaker (Hystrix)

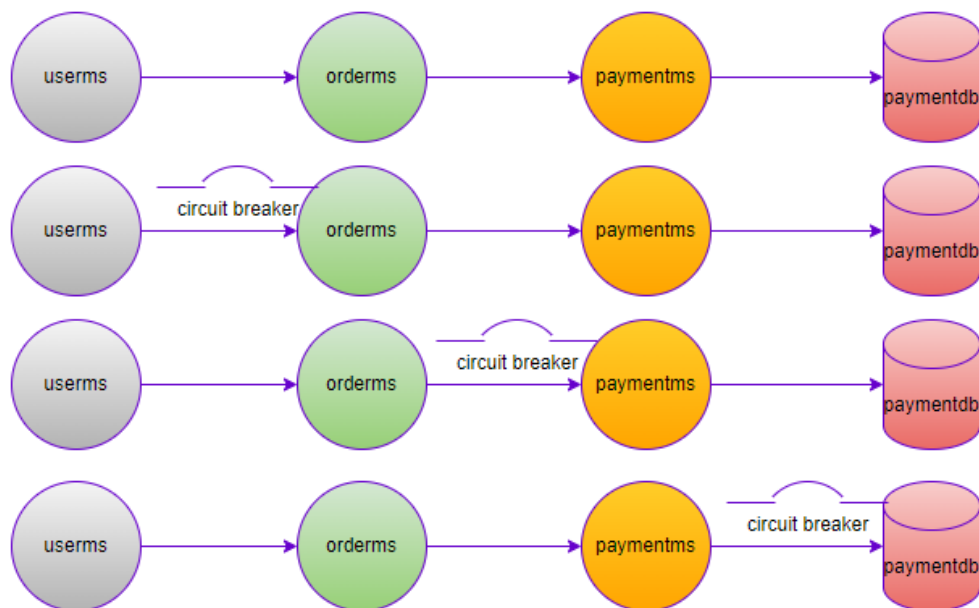
Fallback is a method where defines the behavior of failure.

Once circuit restores, fallback method will not be called. **Hystrix** provides the Resiliency for our application not the HA. **Hystrix works with Ribbon and Eureka.** Ribbon is .jar, used by caller ms, wrap your network calls over the Hystrix. It acts a Fault Tolerant, wherever you are calling other ms wrap over there

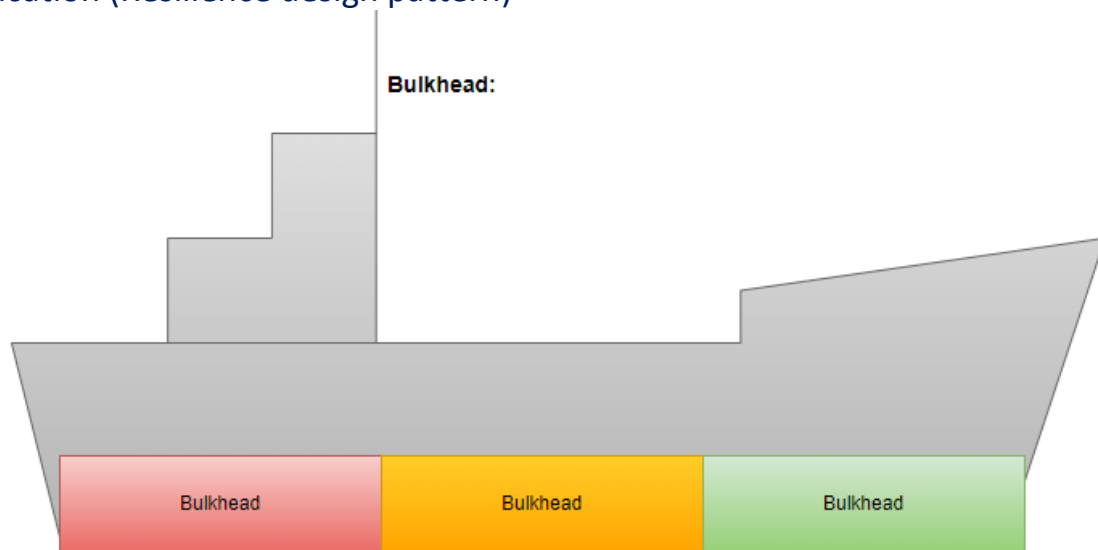
Hystrix: Works on network failure, for other failure you have to do Exception Handling

```
@HystrixCommand(fallbackMethod = "getAllOrdersFromFallback")
public Object getAllOrders() {
    return restTemplate.getForObject( url: "http://orderms/orders", Object.class);
}

public Object getAllOrdersFromFallback() {
    return Arrays.asList("A", "B", "C");
}
```



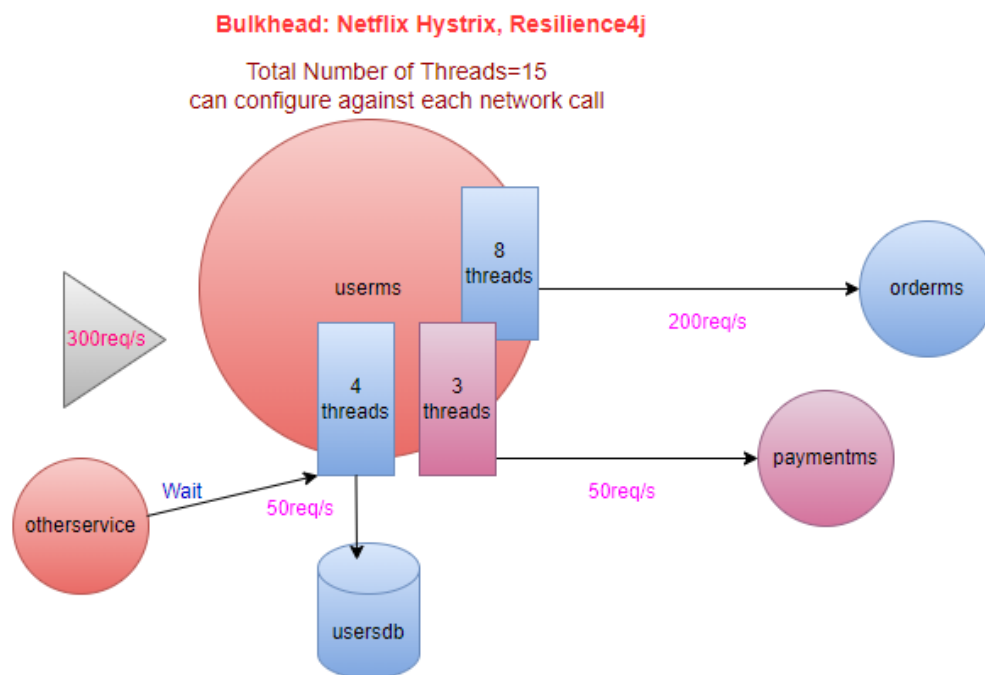
Bulkhead design pattern: due to issue in one part will not impact the whole application (Resilience design pattern)



How Implement?

Reactive manifesto: End-to-End

WebClient is reactive version of Http Client, Number of threads are very less, and threads are not blocked (Asynchronous), there is no message broker.



```

@Override
@HystrixCommand(fallbackMethod = "getUsersFromFallback", commandProperties =
    {
        @HystrixProperty(name = "execution.isolation.thread.timeoutInMilliseconds", value = "30000"),
    },
    threadPoolKey = "getAllUsersThreadPool",
    threadPoolProperties = {
        @HystrixProperty(name = "coreSize", value = "30"),
        @HystrixProperty(name = "maxQueueSize", value = "10")
    })
public List<User> getAllUsers(Optional<String> name) {
    LOGGER.info("*****");
    LOGGER.info("getting all users");
    LOGGER.debug("***");
    LOGGER.error("^^^");
    randomlyRunLong();
}

```

UC:

API Gateway

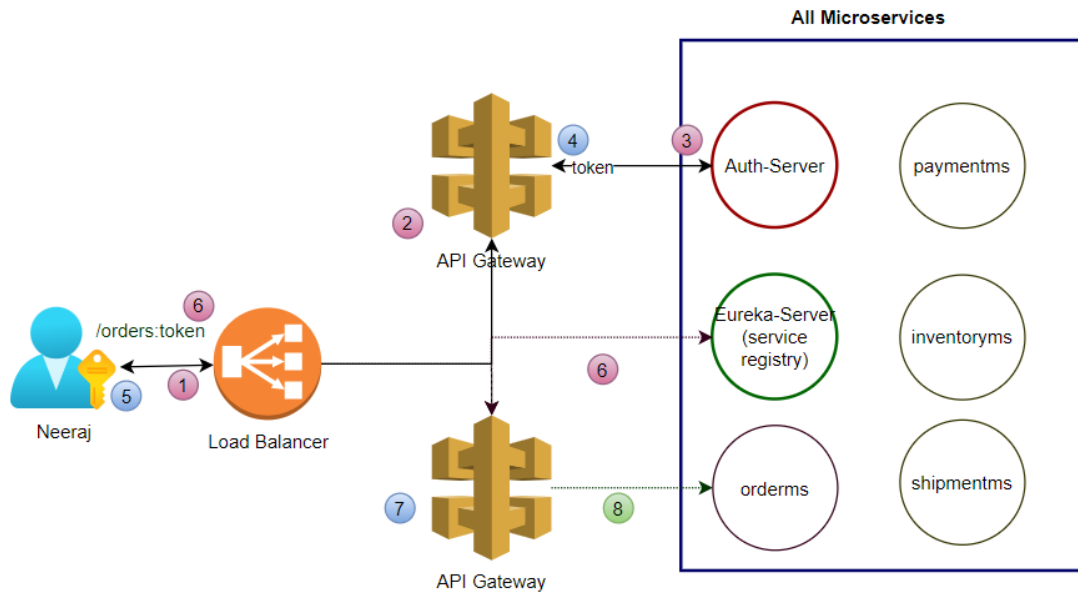
- NFR (Security/Authentication/Caching/URL-hiding.)
- Every cross-cutting concern moves to API Gateway
- API Gateway is for external user, Eureka is for internal components
- Single point of entry in your application
- Must be stateless
- No business logic
- API can have a logic of token validation
- If token is Invalid ApiGateway rejects the request, will not forward it to Eureka/other microservice.
- URL hiding
- Rate Limit (Throttling)
- Denial of Service (Dos)

API Gateway: (Edge Service) Netflix Zuul

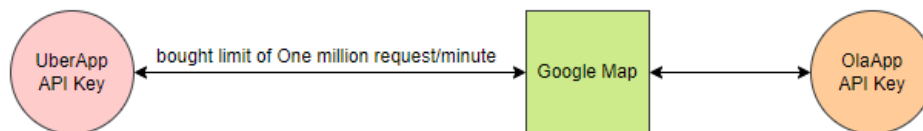
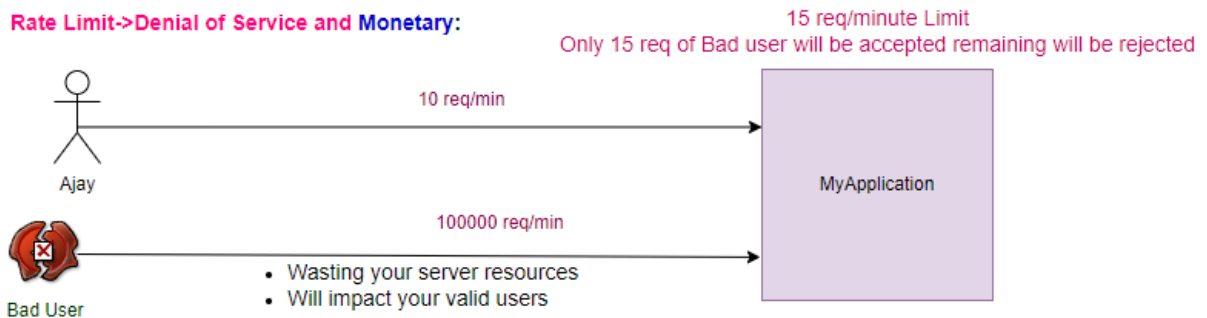
Cross-Cutting concern: Service Mesh (common across most of the services at commonplace)

1. Security: Authorization/Authentication, URL Hiding, Rate Limit
2. Distributed Tracking
3. Audit/Metric Usage: Time is taken by request (duration of request)
4. Clients(Web,Mobile,IOT): specific data
5. Proxy(Entry Point)

URL Hiding-><http://myecommerce.com/api/v1/someservice/users> --> usersms/abc/def/users



Rate Limit->Denial of Service and Monetary:



Throttling: Control the limit bought by the user

RateLimit at ApiGateway level.

Solution: Zuul-Netflix (run on port 8080)

Zuul API comes with the route's URL by default.

localhost:8080//actuator/routes

Whatever eureka sees, ApiGateway can also see same (usersms and ordersms).

Access usersms and ordersms through ApiGateway

localhost:8080/<name in eureka usersms>/users

localhost:8080/<name in eureka ordersms>/orders

UC: We don't want that apigateway access all the routes?

We have to change the default behavior of apigateway

zuul.ignored-services=orderms

zuul.ignored-services=userms

UC: How we do the URL-Hiding?

Url-hiding (orderms by default will not be exposed, exposed using url-hiding)

zuul.ignored-services=orderms

zuul.routes.<ordersid>.serviceId=orderms #third part ordersid is configurable

zuul.routes.ordersid.path=/dummy-orderms/**

#http://localhost:8080/dummy-orderms/orders

Url-hiding (userms by default will not be exposed, exposed using url-hiding)

zuul.ignored-services=userms

zuul.routes.<usersid>.serviceId=userms

zuul.routes.<usersid>.path=/dummy-userms/**

#http://localhost:8080/dumy-userms/users

UC: How Zuul do the Load Balancing

Internally apiGateway (Zuul) uses ribbon as a client-side load balancing.

Cross Cutting Concern- Service Mesh (common across most of the services at common place).

ApiGateway is only for external calls.

server.servlet.context-path=myuserms

appended in the path

UC: How to manage certificates?

We have to provide some TLS setting at apigateway (Zuul).

UC: Filters Using Zuul?

Filter: If you want to add some additional information (uses as a proxy to the ms).

request -> LB-> API Gateway (multiple for HA)

step1: ApiGateway first checks request is for dummy-userms, is there any entry of dummy-userms

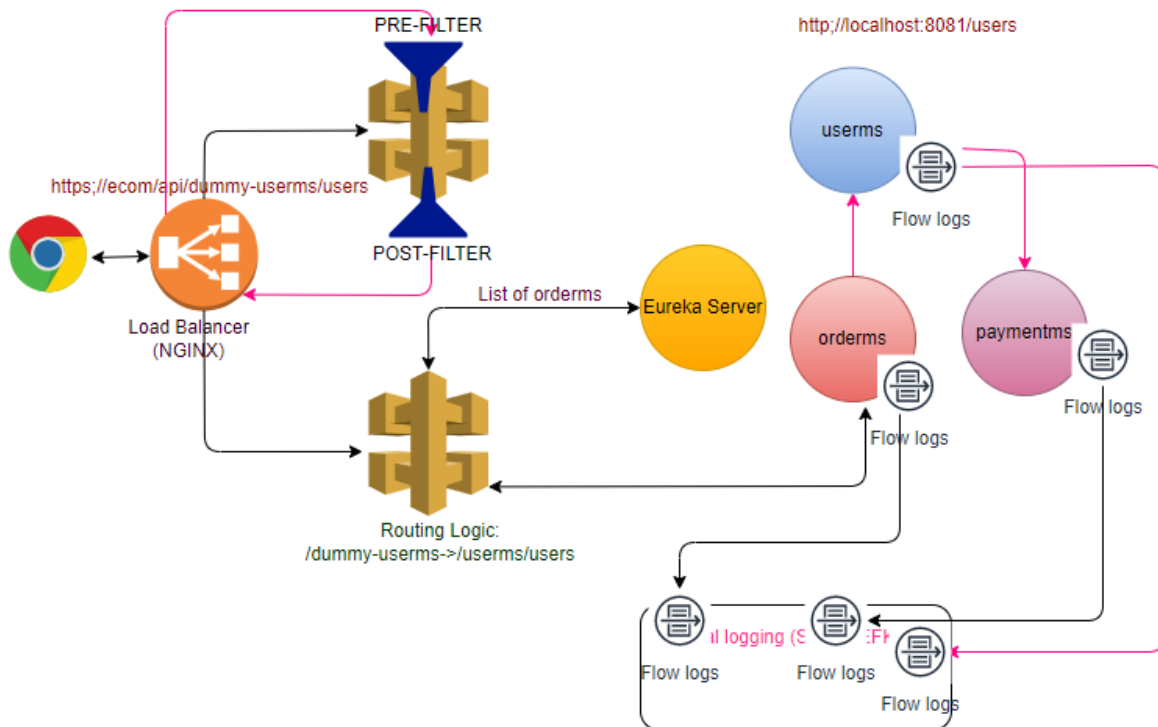
step2: ApiGateway internally uses the Ribbon for client-side load balancing.

step3: Internally uses the Circuit breaker to maintain the resilience.

API Gateway: (Edge Service) Netflix Zuul

Cross Cutting concern: Service Mesh (common across most of the services at common place)

1. Security: Authorization/Authentication, URL Hiding, Rate Limit
2. Distributed Tracking
3. Audit/Metric Usage: Time taken by request
4. Clients(Web,Mobile,IOT): specific data
5. Proxy(Entry Point)



Note: Load Balancer (NGINX) is only for external services, Ribbon Load Balancer is not for external service, just for internal client-side load balancing.

step1: create a pre filter class within the filter package which implements the ZuulFilter

- 1) filterType: before or after (pre or post)
- 2) filterOrder: If there are more than one filter of same type then in which order it will be executed.
- 3) shouldFilter: At **which condition** filter will be executed.
- 4) run: What filter will do (actual logic)

```

public class AddRequestHeaderFilter extends ZuulFilter {
    private static final Logger LOGGER = LoggerFactory.getLogger(AddRequestHeaderFilter.class);
    @Override
    public String filterType() { return "pre"; } // post, error, route

    @Override
    public int filterOrder() { return 0; }

    @Override
    public boolean shouldFilter() {
        // can have custom condition to apply filter or not
        // senderType is queryParams in the request
        // RequestContext.getCurrentContext().getParameter("senderType"); == mobile-client
        return true;
    }

    @Override
    public Object run() throws ZuulException {
        // add starttime to context as Instant.now in prefilter
    }
}

```

step2: create a post filter class within the filter package which implements the ZuulFilter

```

@Override
public Object run() throws ZuulException {
    System.out.println("*****");
    RequestContext currentContext = RequestContext.getCurrentContext();
    final Instant startTime = (Instant)currentContext.get("startTime");

    final long requestDuration = ChronoUnit.MILLIS.between(startTime, Instant.now());
    System.out.println("$$$$$$$$$$$");
    System.out.println("time taken by request: " + requestDuration);
}

```

step3: add the @Bean of both the filters in apiGatewayApplication

Types of Filters:

pre

post

error

@Bean: Is the way of initializing your java class, we delegate this work to spring framework.

```
@SpringBootApplication
@EnableZuulProxy
public class ApigatewayApplication {

    @Bean
    public AddRequestHeaderFilter addRequestHeaderFilter() {
        return new AddRequestHeaderFilter();
    }

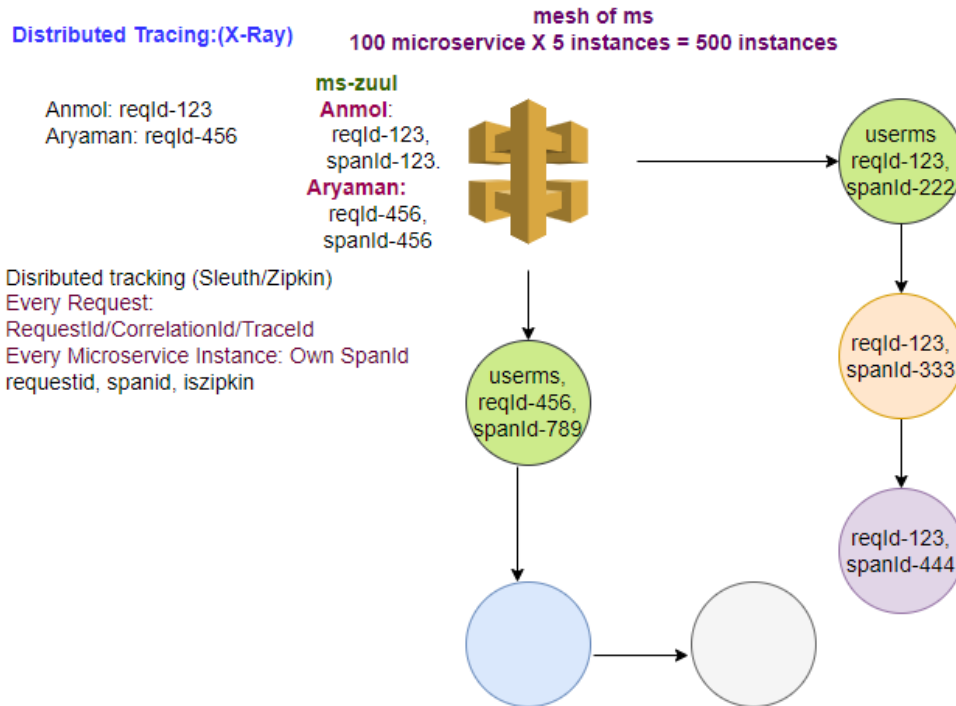
    @Bean
    public PostRequestHeaderFilter postRequestHeaderFilter() {
        return new PostRequestHeaderFilter();
    }
}
```

Note: First request takes more time why?

First time apigateway (Zuul) reads from eureka after that it **cache** the details of Eureka, and subsequently directly calls the service (caching).

Distributed Tracking: Sleuth

UC: money is deducted from the account, but order still showing in pending state. There is a mesh of microservices, for each request we want to know where request gone (complete request flow).



Sleuth Logs of tracing the request?

```

2020-08-30 21:52:46.707 INFO [apigateway,b4cle52006832e94,b4cle52006832e94,true] 9184 --- [nio-
2020-08-30 21:52:46.711 INFO [apigateway,b4cle52006832e94,b4cle52006832e94,true] 9184 --- [nio-

[serviceName, requestId, traceId, boolean-zipkin]

2020-08-30 21:52:46.753 INFO [usersms,b4cle52006832e94,1a2bb62d4d3bd82e,true] 392 --- [nio-8081-
2020-08-30 21:52:46.753 INFO [usersms,b4cle52006832e94,1a2bb62d4d3bd82e,true] 392 --- [nio-8081-
2020-08-30 21:55:04.053 INFO [usersms,b4cle52006832e94,1a2bb62d4d3bd82e,true] 392 --- [nio-8081-

```

Arrays:

First element: name of the ms

Second element: request Id (same for the request): **generated by apigateway**

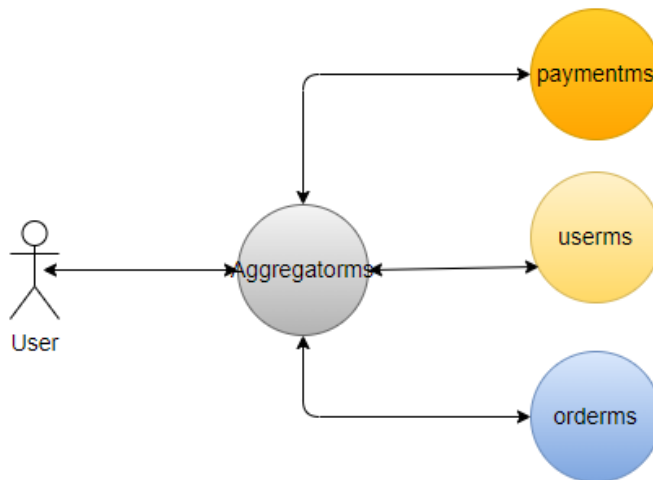
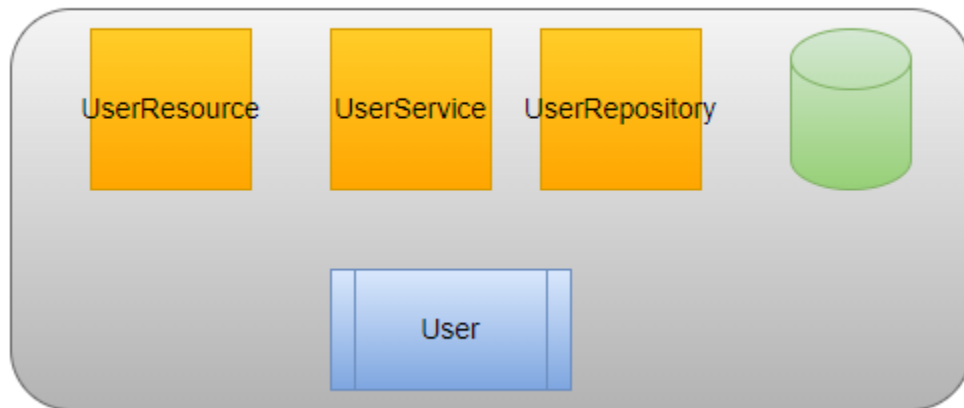
Third element: trace Id (specific to the ms instance in same request)

Fourth element: Whether we export these logs to Zipkin or not?

appending to the log. In real time check the traffic. Troubleshooting the issue.
logs are distributed at ms locations, can be moved to central location.

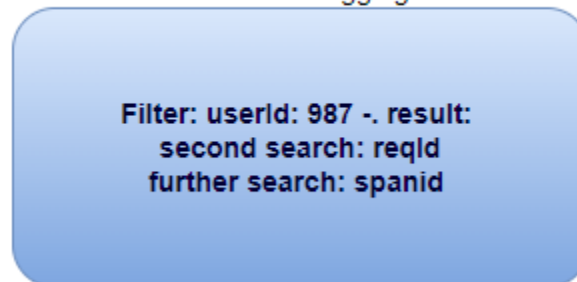
Spring Data Rest (HATEOS)

Hypermedia as the Engine of Application State



Centralized logging (EFK, Splunk)

Centralized Logging:



catalogms

mvn clean install -DskipTests

localhost:8080

localhost:8080/catalogues

localhost:8080/catalogues?size=1

localhost:8080/catalogues?page=1&size=1

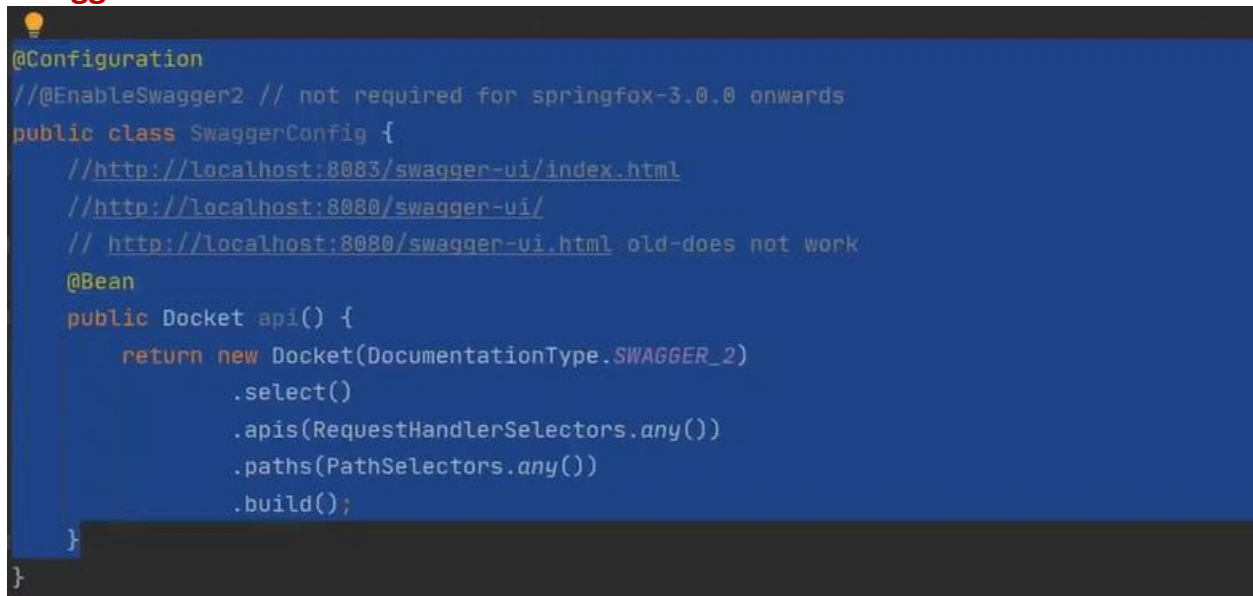
Based on the repository it automatically creates all the CRUD operations.

Its consumer driven API, consumer just use reference URL to navigate.

Limitation: It takes fully control; we can't write service logic.

IDEMPOTENT

Swagger:

A screenshot of a code editor with a dark background and blue syntax highlighting. The code is for a Spring Boot application configuration. It starts with a lightbulb icon in the top left corner. The code defines a configuration class for Swagger, including annotations like @Configuration, @EnableSwagger2, and @Bean, and methods for setting up the Swagger Docket.

```
@Configuration
//@EnableSwagger2 // not required for springfox-3.0.0 onwards
public class SwaggerConfig {
    //http://localhost:8083/swagger-ui/index.html
    //http://localhost:8080/swagger-ui/
    // http://localhost:8080/swagger-ui.html old-does not work
    @Bean
    public Docket api() {
        return new Docket(DocumentationType.SWAGGER_2)
            .select()
            .apis(RequestHandlerSelectors.any())
            .paths(PathSelectors.any())
            .build();
    }
}
```